LAB 3: COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Problem Statement: Remote Patient Vitals Alert & Monitoring App

Problem No: 16

Name: Akshar A. Vaghasiya

SRN: PES1UG25CS852

Section: B

Step 1: Requirements & Challenges

Functional Requirements

- **FR-001:** The system shall continuously ingest patient vital telemetry including SpO2, Heart Rate and Blood Pressure.
- **FR-002:** The system shall continuously evaluate patient vitals against configured clinical thresholds.
- **FR-003:** The system shall identify and flag critical vital-sign anomalies within 2 seconds.
- **FR-004:** The system shall escalate emergency alerts to the appropriate on-call caregiver according to the caregiver matrix.
- **FR-005:** The system shall store patient vital readings and alert information for monitoring and later reference.

Non-Functional Requirements

- **NFR-001 - Performance -** Critical anomalies shall be detected and flagged within 2 seconds.
- **NFR-002 - Availability/Scalability -** The telemetry gateway shall support at least 500 concurrent telemetry streams with 99.99% uptime.

Challenges

- **Performance:** Vital anomalies must be detected within 2 seconds.
- **Scalability:** The system must support at least 500 concurrent telemetry streams.
- **Reliability:** The system requires 99.99% uptime.
- **Security:** Patient health telemetry and emergency alerts must be protected.
- **Real-time processing:** Continuous vital data must be processed without significant delay.
- **Alert reliability:** Critical alerts must reach the correct on-call caregiver.

Step 2 — Compare the 3 Architectures

Layered

- **Advantages**
  - Simple, clear separation, easier maintenance
- **Disadvantages**
  - Layer traversal can introduce delay; scaling individual parts is harder
- **Suitability**
  - Good, but not ideal

Microservices

- **Advantages**
  - Independent scaling, fault isolation, real-time services can be separated
- **Disadvantages**
  - More operational complexity
- **Suitability**
  - Best choice

Client-Server

- **Advantages**
  - Simple deployment and centralized control
- **Disadvantages**
  - Bottleneck and single point of failure
- **Suitability**
  - Less suitable

Architecture I recommend: MICROservices

For **your actual problem statement**, Microservices is the strongest choice.

Why?

Your system needs:

- 500+ concurrent telemetry streams
- anomaly detection within 2 seconds
- 99.99% uptime
- emergency alert escalation
- reliable processing

These requirements make **scalability + fault isolation** particularly important.

Step 3 — Exactly 5 Components

Component 1 — Telemetry Ingestion Gateway

Responsibility:
Receives continuous SpO2, Heart Rate and Blood Pressure data from patient monitoring devices.

Component 2 — Vitals Monitoring Engine

Responsibility:
Processes incoming telemetry and compares vital readings against configured clinical thresholds.

Component 3 — Alert Escalation Service

Responsibility:
Generates emergency alerts and determines the appropriate caregiver using the caregiver escalation matrix.

Component 4 — Patient/Caregiver Dashboard

Responsibility:
Displays patient vital information, anomaly status and emergency alerts to authorized users.

Component 5 — Patient Data Store

Responsibility:
Stores patient telemetry, threshold configurations and alert history.

Exactly 4 Interfaces

Interface 1

Telemetry Ingestion Gateway ↔ Vitals Monitoring Engine

Label:

Vitals Data Interface — Streaming API

Purpose:

Sends normalized patient vital readings for real-time analysis.

Interface 2

Vitals Monitoring Engine ↔ Alert Escalation Service

Label:

Alert Trigger Interface — Internal API

Purpose:

Sends critical anomaly information to the alert service

Interface 3

Vitals Monitoring Engine ↔ Patient Data Store

Label:

Patient Data Interface — Database Queries

Purpose:

Stores/retrieves patient vital readings and threshold information.

Interface 4

Alert Escalation Service ↔ Patient/Caregiver Dashboard

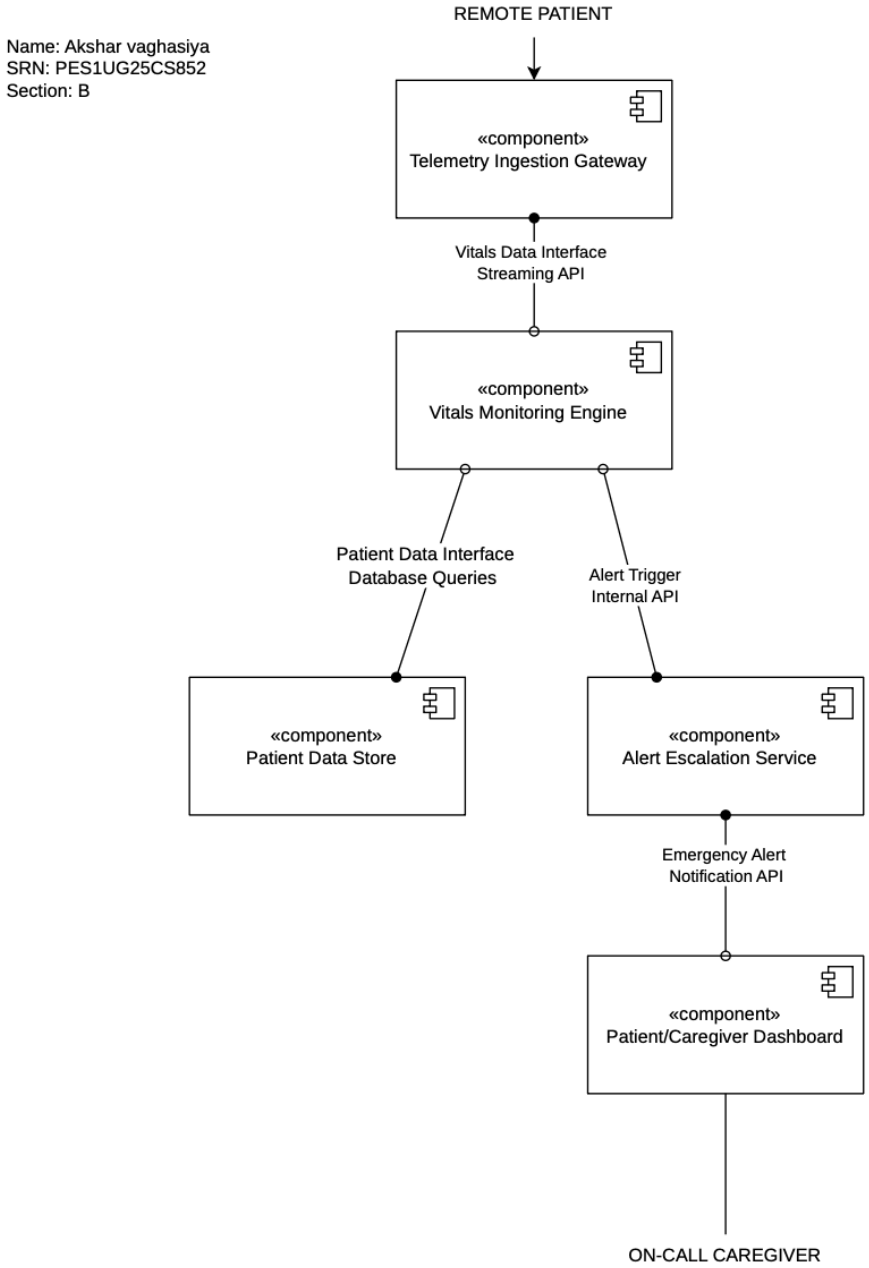
Label:

Emergency Alert Interface — Notification API

Purpose:

Delivers emergency alerts and escalation status to authorized caregivers.

Component Diagram



ARCHITECTURAL JUSTIFICATION

Remote Patient Vitals Alert & Monitoring App

Architecture Selection — Microservices Architecture

We chose **Microservices Architecture** for the Remote Patient Vitals Alert & Monitoring App because the system requires continuous telemetry processing, rapid anomaly detection, high availability, and support for multiple concurrent patient monitoring streams.

Reason 1 — Scalability:

The system must support at least **500 concurrent continuous telemetry streams**. Microservices architecture allows components such as telemetry ingestion and vital monitoring to be scaled independently when the number of monitored patients increases. This makes the architecture suitable for handling continuous patient data at scale.

Reason 2 — Fault Isolation and Reliability:

The system requires **99.99% uptime** and reliable emergency alert processing. Microservices separate major system functions into independent services. Therefore, a failure in one service can be isolated, reducing the risk of the entire monitoring system becoming unavailable.

Security Advantage:

Patient vital information is sensitive data, so security is important. Microservices allow authentication and authorization to be applied at service boundaries. Communication between services can also be secured, helping prevent unauthorized access to patient telemetry and alert information.

Performance Benefit:

The system must detect and flag critical anomalies within **2 seconds**. The Vitals Monitoring Engine can process telemetry independently and can be scaled when required. This helps reduce processing delays and supports the system's real-time anomaly detection requirement.

Conclusion:

Microservices Architecture is the most suitable choice because it provides **independent scalability, fault isolation, and improved support for real-time processing**, which directly address the requirements of the Remote Patient Vitals Alert & Monitoring App.